

7. 外设控制基础

7. 外设控制基础

7.2 ESP-NOW 介绍

- 7.1.1 ESP-NOW概述
- 7.1.2 工作原理
- 7.1.3 通信模式
- 7.1.4 数据结构与限制
- 7.1.5 安全机制

7.2 程序下载说明（必看）

7.3 ESP-NOW控制

- 7.3.1 项目介绍
- 7.3.2 实现流程
- 7.3.3 程序下载
- 7.3.4 实现效果
- 7.3.5 程序分析

7.4 传送带控制

- 7.4.1 项目介绍
- 7.4.2 实现流程
- 7.4.3 程序下载
- 7.4.4 实现效果
- 7.4.5 程序分析

7.5 滑轨控制

- 7.5.1 项目介绍
- 7.5.2 实现流程
- 7.5.3 程序下载
- 7.5.4 实现效果
- 7.5.5 程序分析

7.6 电机速度控制

- 7.6.1 项目介绍
- 7.6.2 实现流程
- 7.6.3 程序下载
- 7.6.4 实现效果
- 7.6.5 程序分析

7.7 履带底盘控制

- 7.7.1 项目介绍
- 7.7.2 实现流程
- 7.7.3 程序下载
- 7.7.4 实现效果
- 7.7.5 程序分析

7.8 麦轮底盘控制

- 7.8.1 项目介绍
- 7.8.2 实现流程
- 7.8.3 程序下载
- 7.8.4 实现效果
- 7.8.5 程序分析

7.9 同步器摇杆控制

- 7.9.1 项目介绍
- 7.9.2 实现流程
- 7.9.3 程序下载
- 7.9.4 实现效果

7.9.5 程序分析

7.10 同步器摇杆控制移动底盘

7.10.1 项目介绍

7.10.2 实现流程

7.10.3 程序下载

7.10.4 实现效果

7.10.5 程序分析

Note

本章节的玩法并不适用于NexArm所有版本，在学习章节课程之前，须先确定有无对应的外设。

7.2 ESP-NOW 介绍

7.1.1 ESP-NOW概述

ESP-NOW 是乐鑫科技（Espressif）推出的一种基于 Wi-Fi 物理层与数据链路层的**无连接、低功耗、短数据通信**协议。与传统 Wi-Fi 通信不同，ESP-NOW 不依赖路由器，也无需建立连接或获取 IP 地址，而是通过设备的 **MAC 地址** 实现点对点或一对多的直接通信。



由于省去了扫描网络、关联和 IP 分配等过程，ESP-NOW 具有**毫秒级低延迟**和较低功耗的特点，非常适用于电池供电的无线控制、传感器数据传输以及实时性要求较高的遥控场景。尽管其单次数据包大小限制在 **250 字节以内**，但其支持的 **AES 加密机制**能够有效保障通信安全，是一种在通信距离、响应速度与功耗之间取得良好平衡的无线通信方案。

7.1.2 工作原理

ESP-NOW 的底层仍然工作在 **2.4GHz Wi-Fi 物理层** 上，但它并不经过传统的 TCP/IP 协议栈，也不需要像普通 Wi-Fi 那样先完成扫描热点、连接路由器、获取 IP 地址等流程，而是直接基于 **802.11 数据帧** 完成设备之间的短数据通信。发送端只需知道目标设备的 **MAC 地址**，即可将数据封装后直接发送到对端，接收端在收到帧后再进行解析与处理。



其基本工作流程如下：首先由主控芯片初始化 Wi-Fi 模块并设置工作信道，然后通过 `esp_now_add_peer()` 添加通信对端，建立对等节点信息；之后发送端调用 `esp_now_send()` 将数据帧发送给指定 MAC 地址或广播地址，接收端则通过注册的回调函数实时获取接收到的数据内容。当应用层需要进行状态同步、按键控制或数据透传时，只需在接收回调中解析数据并执行对应操作即可。

相较于传统 Wi-Fi 通信，ESP-NOW 省略了连接建立过程，因此具有更快的响应速度。特别是在机械臂控制、遥控器同步、状态广播等场景中，这种“初始化后即可直接收发”的机制能够明显降低通信延迟，提高控制的实时性与稳定性。

7.1.3 通信模式

ESP-NOW 支持多种常用通信模式，既可以实现 **点对点通信**，也可以实现 **一对多广播通信**。在点对点模式下，发送端将数据发送给某一个固定的目标 MAC 地址，适合用于遥控器与单台设备之间的定向控制。此时通信链路明确，数据管理简单，适合对可靠性要求较高的控制任务。



在一对多模式下，发送端可以使用 **广播 MAC 地址** 将同一份数据发送给多个设备。多个接收端只要工作在相同信道并处于可接收状态，就可以同时收到该数据。这种方式常用于多设备同步控制，例如同步器同时向多个机械臂下发动作指令，或者主控节点统一广播状态与控制信息。

除了单向发送外，ESP-NOW 还支持 **双向通信**。也就是说，设备既可以作为发送端，也可以作为接收端，双方都可以互相收发数据。通过这种方式，可以实现“主控发送控制命令，从机回传执行状态”的闭环控制结构。在实际应用中，可根据控制需求选择单播、广播或双向交互等不同模式，以兼顾实时性、灵活性与系统复杂度。

7.1.4 数据结构与限制

ESP-NOW 传输的数据本质上是 **用户自定义的字节数组**，因此开发者可以根据自己的应用场景设计通信协议。例如，可以将按键状态、摇杆值、坐标数据、舵机位置、控制命令等内容封装到结构体或字节流中，再通过 ESP-NOW 进行发送。接收端只需按照约定的数据格式进行解析，就能够正确还原出发送端的控制信息。

PAYLOAD

结构可自定义

开发者可以把结构体或命令参数打包成字节流，再由接收端按协议解包。

LIMIT

单帧长度受限

常规应用中单帧有效载荷应控制在 250 字节以内，超出时需要拆包。

CHANNEL

必须同信道

通信双方应工作在同一 Wi-Fi 信道，否则即使 MAC 地址正确也无法正常收发。

ADVICE

适合短小实时数据

应尽量减少无效字段，优先传输轻量、高频、低延迟的控制类数据。

在使用过程中，需要注意 ESP-NOW 对单帧数据长度存在限制。常规情况下，单次发送的有效载荷长度应控制在 **250 字节以内**。如果超过该范围，数据就需要拆包发送，否则会导致发送失败或数据异常。此外，由于 ESP-NOW 工作在固定信道上，通信双方必须处于 **相同 Wi-Fi 信道**，否则即使 MAC 地址正确，也无法正常收发数据。

另外，ESP-NOW 更适合传输 **短小、实时性强** 的数据，而不适合大文件或长数据流的连续传输。对于高频控制类应用，建议尽量简化数据结构，减少无效字段，降低每次发送的数据量，这样能够有效减小传输延迟，提升通信效率与系统响应速度。

7.1.5 安全机制

ESP-NOW 支持一定程度的通信安全保护，其核心机制是基于 **AES-128 加密算法** 对数据进行加密传输。开发者可以为设备配置 **主密钥（PMK）** 与 **本地主密钥（LMK）**，在启用加密后，只有拥有正确密钥的设备之间才能完成有效通信，从而避免普通设备随意接收或伪造数据。



在实际应用中，如果只是进行实验测试或局域环境下的功能验证，也可以先使用未加密的广播方式进行通信，这种方式配置简单，调试方便；而在涉及正式控制、设备联动或存在外部干扰风险的场景中，则建议优先使用配对通信并开启加密功能，以提高通信的安全性及可靠性。

需要注意的是，ESP-NOW 的安全机制虽然可以对数据传输过程进行保护，但通信安全不仅仅依赖加密本身，还应结合 **固定通信信道**、**限定对端 MAC**、**合理设计数据协议** 等措施共同使用。只有在链路配置、数据格式和设备管理都较为规范的前提下，才能更好地保证整个无线控制系统的稳定运行。

7.2 程序下载说明（必看）

Note

NexArm 配套外设的组装教程可前往本文档同目录下的“\03 组装视频”文件夹中查看。

1. 使用 Type-C 数据线将 NexArm 与电脑进行连接。



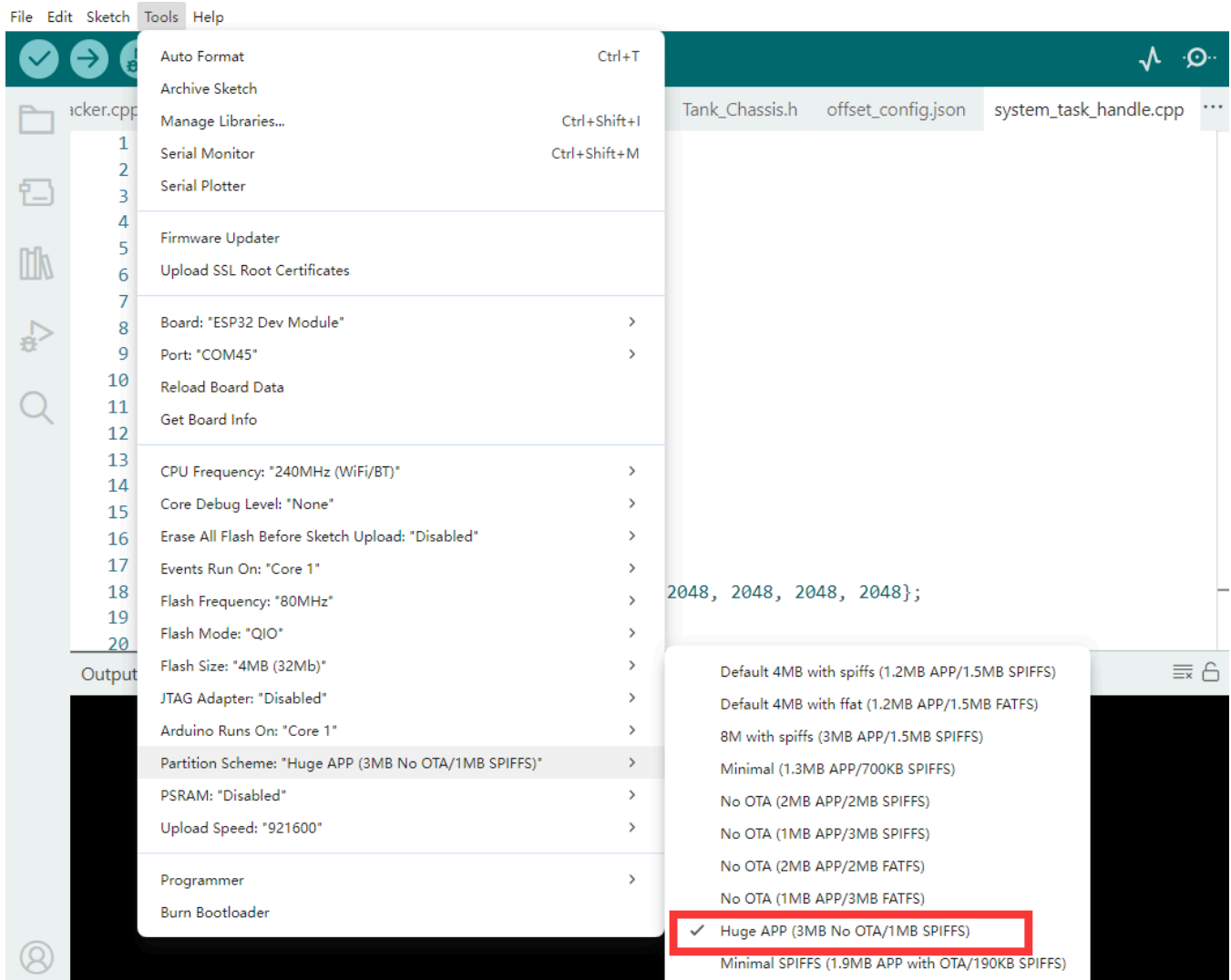
2. 打开本文档同路径下"源码"中对应的ino程序文件。

 Nex_Arm.ino

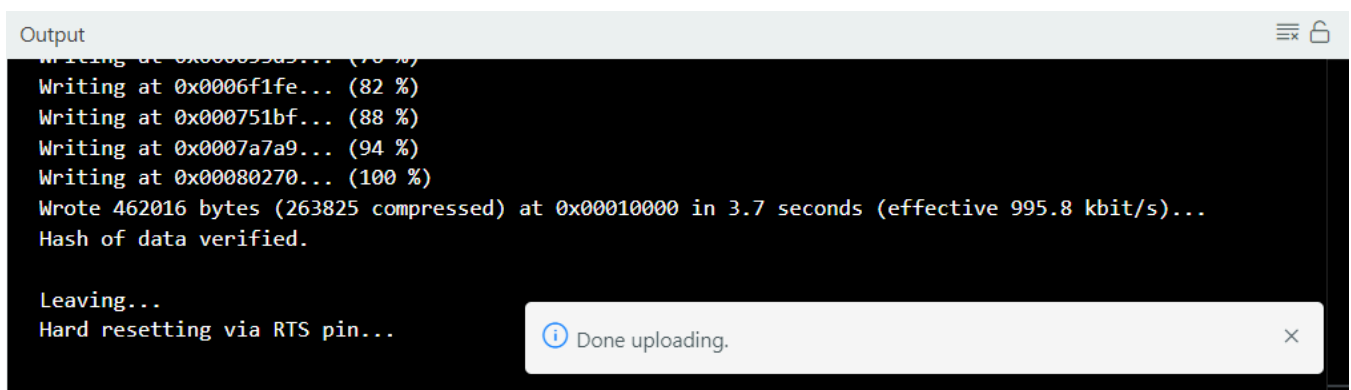
3. 打开后，选择开发板型号，具体型号如下图所示：



4. 点击菜单栏的"工具"，按照下图选择相应的ESP32的开发板配置：



5. 先点击"编译", 再点击"上传", 当软件下方输出框出现如下界面, 说明程序下载成功:

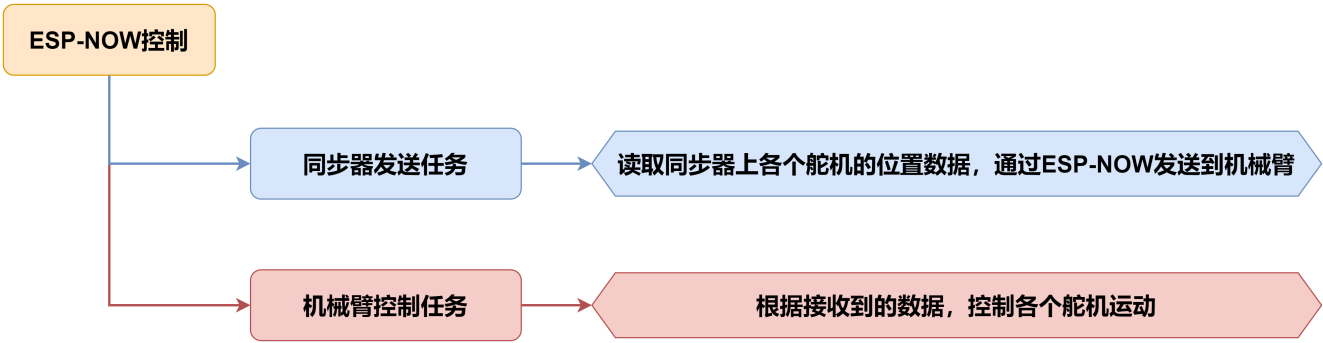


7.3 ESP-NOW控制

7.3.1 项目介绍

本门课程通过 ESP-NOW 实现机械臂与同步器之间的无线控制。同步器程序负责读取 6 路舵机当前位置, 并将这些位置数据通过 ESP-NOW 周期发送给目标机械臂; 机械臂程序负责接收同步数据、完成舵机映射和动作执行, 同时支持同步开关控制与 OLED 状态显示。

7.3.2 实现流程



7.3.3 程序下载

- 机械臂程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“同步器控制\NexArm程序”程序进行下载。该程序用于接收同步器发送的 ESP-NOW 数据，并控制机械臂执行同步动作。具体下载步骤可参考[7.2 程序下载说明（必看）](#)。

- 同步器程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“同步器控制\同步器（示教器）程序”程序进行下载。具体下载步骤可参考[7.2 程序下载说明（必看）](#)。

7.3.4 实现效果

两端程序分别下载完成后，上电进入同一 ESP-NOW 信道。同步器端会持续读取自身 6 路舵机位置并发送给外部设备；机械臂端在按下 KEY1 按键开启同步模式后，会接收这些舵机位置数据并完成本机舵机联动。机械臂 OLED 会实时显示当前末端位姿，同步器串口则会输出当前发送的舵机位置数据，便于观察无线同步效果。

7.3.5 程序分析

- 机械臂分析

1. 机械臂程序先定义了 ESP-NOW 控制需要的运行时状态，包括协议解析对象、参数存储对象、当前信道、全局加速度、同步开关、OLED 刷新时间、电池刷新时间以及最近一次舵机反馈时间。这些变量共同决定了无线同步运行时的基础配置和状态缓存。

```
CommProtocol_t at32_protocol;
Preferences prefs;

uint8_t global_channel = DEFAULT_ESPNOW_CHANNEL;
uint8_t global_acc = 245;
bool espnow_sync_enabled = false;

uint32_t last_oled_refresh_ms = 0;
uint32_t last_bat_refresh_ms = 0;
uint32_t button_enable_ms = 0;

int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
uint32_t g_last_servo_feedback_ms = 0;
```

2. `load_config()` 和 `save_config()` 负责保存和恢复 ESP-NOW 相关参数，例如当前信道和同步加速度。这样设备在重新上电后仍然可以保持之前设置过的通信参数，不需要每次重新配置。

```
void save_config()
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.end();
}

void load_config()
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", DEFAULT_ESPNOW_CHANNEL);
    global_acc = prefs.getUChar("acc", 245);
    prefs.end();
}
```

3. `restart_espnw()` 用于按当前信道重新启动 ESP-NOW，并在启动成功后重新注册协议回调，同时按当前同步状态设置从机是否允许执行同步动作。这样无线模块初始化与同步状态控制被统一收口到一个函数里。

```
bool restart_espnw()
{
    espnow_ctrl.stop();
    delay(50);

    if (!espnow_ctrl.begin(global_channel)) {
        arm.board.buzzer.set(120, 80, 1, 1000);
        return false;
    }

    espnow_ctrl.register_ops_callback(func_ctrl_callback);
    apply_follow_sync_state(espnow_sync_enabled, false);

    return true;
}
```

4. `func_ctrl_callback()` 是机械臂端处理 ESP-NOW 协议控制包的核心入口。对于 `CMD_COORDINATE_SET`、`CMD_ARM_MOVE_INC` 等控制命令，程序会先透传给 AT32，再同步更新本地姿态缓存；对于设置通道、同步开关、设置配对 MAC 等命令，则在 ESP32 本地直接处理。

```
void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t payload_len = self->elements.length >= 2 ? (uint8_t)(self->elements.length - 2) : 0;

    if (self->elements.id == 0xFF) {
        switch (self->elements.cmd) {
            case CMD_ESPNOW_SYNC_CTRL:
                if (payload_len >= 1) {
```

```

        apply_follow_sync_state(self->elements.args[0] == 1, true);
    }
    break;

    case CMD_COORDINATE_SET:
        servo.tx_frame_write(0xFF, self->elements.cmd, self->elements.args,
payload_len);
        update_pose_from_coordinate_set(self->elements.args, payload_len);
        break;
    }
}
}
}

```

5. ESP-NOW 的主同步数据并不是普通坐标指令，而是 `ArmPacket_t` 结构体。接收端在 `OnDataRecv()` 中判断收到的数据长度是否等于该结构体长度；若同步模式开启，则先缓存主机发来的 6 路舵机位置，再在主循环中统一执行一次映射和舵机下发，避免在接收回调中直接大量发串口数据。

```

typedef struct __attribute__((packed)) {
    uint32_t seq;
    int16_t pos[6];
} ArmPacket_t;

if (espnw_ctrl.sync_enabled && len == sizeof(ArmPacket_t)) {
    ArmPacket_t *pkt = (ArmPacket_t*)incomingData;
    for (int i = 0; i < 6; ++i) {
        espnw_ctrl.pending_master_pos[i] = pkt->pos[i];
    }
    espnw_ctrl.pending_sync_apply = true;
}

```

6. `mapMasterTeachToFollower()` 和 `applyMasterTeachPositions()` 负责把主机舵机值映射为从机目标值，并统一调用 `servo.sync_write_pos_speed()` 发送给 6 路舵机。其中 2 号和 6 号舵机还加入了方向和范围修正，因此同步过程并不是简单地把原值原样拷贝过去。

```

void EspNow_Ctrl_t::mapMasterTeachToFollower(const int16_t *master_positions, int16_t
*target_positions) {
    for (int i = 0; i < 6; i++) {
        int id = i + 1;
        int16_t master_pos = master_positions[i];

        if (id == 2) {
            target_positions[i] = (int16_t)constrain(4096 - (int32_t)master_pos, 0, 4096);
        } else if (id == 6) {
            int32_t delta = (int32_t)master_pos - 2048;
            target_positions[i] = (int16_t)constrain(2833 + delta * 4, 1195, 2833);
        } else {
            target_positions[i] = master_pos;
        }
    }
}

```

7. 主循环里先处理挂起的同步帧，再处理 AT32 回包、按键、电池和 OLED 刷新。当检测到 `BOOT` 键时，会切换同步模式；当同步模式打开后，机械臂端就会不断处理同步器发来的舵机位置数据，并在 OLED 上显示当前姿态。

```
void system_loop_handler(void)
{
    if (espnow_sync_enabled) {
        espnow_ctrl.processPendingSyncFrame(0);
    }

    pump_at32_feedback();
    arm.board.button.update();
    arm.board.buzzer.update();

    uint32_t now = millis();
    if (now >= button_enable_ms && arm.board.button.is_clicked(0)) {
        toggle_follow_sync();
    }

    if (now - last_oled_refresh_ms >= OLED_REFRESH_INTERVAL_MS) {
        last_oled_refresh_ms = now;
        arm.board.oled.show_status(
            arm.current_pose.x,
            arm.current_pose.y,
            arm.current_pose.z,
            arm.current_pose.pitch,
            arm.current_pose.roll,
            arm.current_pose.claw
        );
    }
}
```

• 同步器分析

1. 同步器程序的核心数据结构是 `ArmPacket_t`，其中包含一个时间序号 `seq` 和 6 路舵机位置 `pos[6]`。每次发送时，程序都会把当前时刻写入 `seq`，并把 6 路舵机读取值写入 `pos[]` 数组，作为一次完整的同步数据包。

```
typedef struct __attribute__((packed)) {
    uint32_t seq;
    int16_t pos[6];
} ArmPacket_t;

int16_t last_valid_pos[6] = {2048, 2048, 2048, 2048, 2048, 2048};
```

2. 初始化阶段先启动串口与总线舵机接口，再配置 Wi-Fi 为 STA 模式并设置固定信道，然后初始化 ESP-NOW 并添加广播地址作为目标节点。这样同步器就会把示教数据直接广播给同信道内的接收端机械臂。

```
wifi.mode(WIFI_STA);
wifi.disconnect();

esp_wifi_set_promiscuous(true);
esp_wifi_set_channel(WIFI_CHANNEL, WIFI_SECOND_CHAN_NONE);
```

```

esp_wifi_set_promiscuous(false);

if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW Init Error");
    return;
}

esp_now_peer_info_t peerInfo = {};
memcpy(peerInfo.peer_addr, broadcastMac, 6);
peerInfo.channel = WIFI_CHANNEL;
peerInfo.encrypt = false;
esp_now_add_peer(&peerInfo);

```

3. 初始化完成后，程序会依次关闭 1~6 号舵机力矩。这样同步器本体在示教时可以被手动拖动，随后主循环再把读到的舵机位置发送给接收端，从而形成“手拖示教、无线同步”的控制方式。

```

for (int i = 1; i <= 6; i++) {
    servo.disable_torque(i);
    delay(5);
}

```

4. 主循环以 `SEND_INTERVAL_MS` 为发送周期，每轮先逐个读取 6 路舵机当前位置。读取成功时就更新 `last_valid_pos[]`，读取失败时则保留上一次的有效值，这样可以避免个别读数异常导致整个同步帧出现明显跳变。

```

for (int i = 0; i < 6; i++) {
    int16_t pos = 0;
    ServoStatus_t status = servo.read_pos(i + 1, &pos);

    if (status.error_bits.bit_rx == 0 && status.error_bits.bit_tx == 0 && pos != 0) {
        last_valid_pos[i] = pos;
    }
    delayMicroseconds(200);
}

```

5. 所有舵机值准备好后，程序把它们写入 `teachPacket.pos[]`，再调用 `esp_now_send()` 广播发送。也就是说，同步器端本身不做任何复杂解析，它的主要职责就是“读取舵机值、封装结构体、周期发送”。

```

for (int i = 0; i < 6; i++) {
    teachPacket.pos[i] = last_valid_pos[i];
}

esp_now_send(broadcastMac, (uint8_t*)&teachPacket, sizeof(ArmPacket_t));

```

6. 同步器端保留了串口调试输出，每隔一段时间就打印当前发送的 6 路舵机位置。通过这些打印信息，可以直接观察同步器当前读取到的示教数据是否正常，也便于和接收端动作进行对照调试。

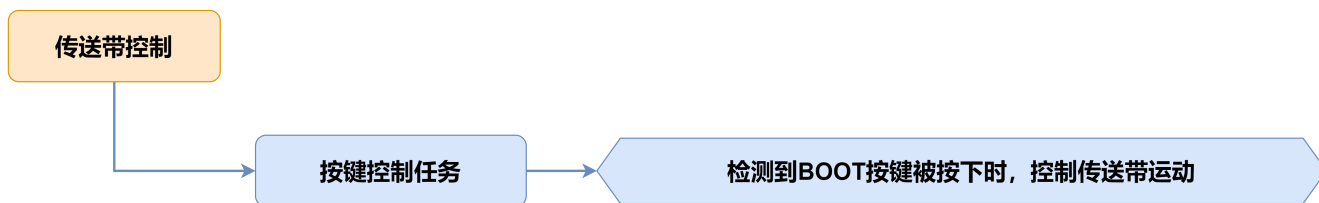
```
if (currentMillis - lastDebugTime > 500) {  
    lastDebugTime = currentMillis;  
    Serial.printf("[Teach] Pos: %d %d %d %d %d %d\n",  
        teachPacket.pos[0], teachPacket.pos[1], teachPacket.pos[2],  
        teachPacket.pos[3], teachPacket.pos[4], teachPacket.pos[5]);  
}
```

7.4 传送带控制

7.4.1 项目介绍

本节课程演示如何通过控制板上的传送带接口实现启停控制。程序上电后先将传送带速度设置为 0，按下 BOOT 键后在速度 100 与 0 之间切换，同时在 OLED 上显示当前运行状态。

7.4.2 实现流程



7.4.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“传送带控制”程序进行下载，具体下载步骤可参考[7.2 程序下载说明](#)（必看）。

7.4.4 实现效果

程序上电后，传送带默认停止。按下 KEY1（BOOT）键后，传送带开始以速度 20 运行；再次按下 KEY1 键后，传送带每次速度加20。按下 KEY2 (USER),传送带停止运动。OLED 会同步显示当前速度和运行状态。

7.4.5 程序分析

1. 程序先引入 Arduino、Wire 和板级封装头文件，然后设置主循环任务栈大小，并定义两个按键编号和传送带 I2C 引脚。这里的 KEY1_ID 和 KEY2_ID 会在主循环中用于区分两个按键，21/22 则用于初始化连接传送带驱动的 wire1 总线。

```
#include "Arduino.h"  
#include "Wire.h"  
#include "Nex_Arm_Board.h"  
  
SET_LOOP_TASK_STACK_SIZE(4096);  
  
#define KEY1_ID 0  
#define KEY2_ID 1  
#define CONVEYOR_I2C_SDA_PIN 21  
#define CONVEYOR_I2C_SCL_PIN 22
```

2. 程序创建了一个 `HW_Board` 板级对象，并定义传送带速度数组。 `next_speed_index` 用来记录下一次 KEY1 应该切换到哪个速度， `current_speed` 保存当前速度， `conveyor_running` 保存运行状态， `oled_ready` 用来避免 OLED 初始化失败后继续刷新显示。

```
static HW_Board board;

static const int8_t conveyor_speeds[] = {20, 40, 60, 80, 100};
static const uint8_t conveyor_speed_count = sizeof(conveyor_speeds) /
sizeof(conveyor_speeds[0]);

static uint8_t next_speed_index = 0;
static int8_t current_speed = 0;
static bool conveyor_running = false;
static bool oled_ready = false;
```

3. `show_conveyor_status()` 负责刷新 OLED 显示。如果 OLED 没有初始化成功就直接返回；初始化成功时，程序会显示传送带标题、当前运行状态和当前速度，使按键操作后的状态可以同步显示出来。

```
static void show_conveyor_status()
{
    if(!oled_ready) return;

    board.oled.set_custom_text(0, "Conveyor Belt");
    board.oled.set_custom_text(1, conveyor_running ? "KEY2 Status: RUN" : "KEY2 Status: STOP");
    board.oled.set_custom_text(2, String("KEY1 Speed: ") + String(current_speed));
    board.oled.set_custom_text(3, "");
    board.oled.show_custom();
}
```

4. `run_next_speed()` 负责处理 KEY1 的速度循环。每次调用时，程序先从速度数组中取出当前索引对应的速度，再把索引加一并取余，实现 20 -> 40 -> 60 -> 80 -> 100 -> 20 的循环，然后把速度写入传送带并刷新 OLED。

```
static void run_next_speed()
{
    current_speed = conveyor_speeds[next_speed_index];
    next_speed_index = (next_speed_index + 1) % conveyor_speed_count;
    conveyor_running = true;

    board.conveyor.set_speed(current_speed);
    show_conveyor_status();
}
```

5. `stop_conveyor()` 负责处理 KEY2 停止功能。它会把当前速度设为 0，把运行状态设为停止，并把下一次速度索引重置为 0，所以传送带停止后再次按 KEY1 会重新从速度 20 开始运行。

```
static void stop_conveyor()
{
    current_speed = 0;
    conveyor_running = false;
    next_speed_index = 0;

    board.conveyor.set_speed(0);
    show_conveyor_status();
}
```

6. 初始化阶段先启动 OLED，再用 `wire1.begin()` 配置传送带 I2C 总线，随后调用 `board.conveyor.begin(&wire1)` 初始化传送带驱动。最后调用 `stop_conveyor()`，确保程序上电后传送带默认停止，并在 OLED 上显示停止状态。

```
void setup()
{
    oled_ready = board.oled.begin();

    wire1.begin(CONVEYOR_I2C_SDA_PIN, CONVEYOR_I2C_SCL_PIN, 100000);
    board.conveyor.begin(&wire1);

    stop_conveyor();
}
```

7. 主循环中持续刷新按键状态。检测到 KEY1 点击时调用 `run_next_speed()` 切换到下一档速度；检测到 KEY2 点击时调用 `stop_conveyor()` 停止传送带。最后的 `delay(10)` 用于降低循环频率，让按键轮询保持稳定。

```
void loop()
{
    board.button.update();

    if(board.button.is_clicked(KEY1_ID)) {
        run_next_speed();
    }

    if(board.button.is_clicked(KEY2_ID)) {
        stop_conveyor();
    }

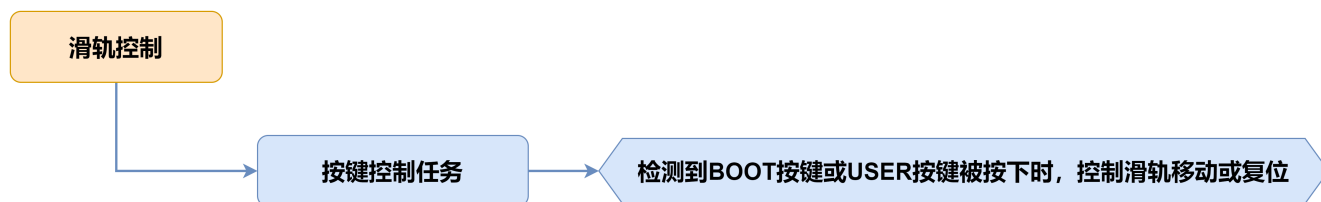
    delay(10);
}
```

7.5 滑轨控制

7.5.1 项目介绍

本节课程演示如何通过步进滑轨接口控制滑轨移动指定步数。程序上电后先把滑轨细分模式设置为 `0x02`，随后执行一次回零操作。按下 `BOOT` 键后，滑轨向前移动 `1000` 步；按下 `USER` 键后，滑轨重新回到初始位置。

7.5.2 实现流程



7.5.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“滑轨步数控制”程序进行下载，具体下载步骤可参考[7.2 程序下载说明（必看）](#)。

7.5.4 实现效果

程序上电后，滑轨先完成细分模式配置和回零。按下 `BOOT` 键后，滑轨向前移动 `1000` 步；按下 `USER` 键后，滑轨执行回零。OLED 会显示当前细分模式、步数信息以及最近一次按键动作。

7.5.5 程序分析

1. 程序首先定义了滑轨细分模式、单次移动步数、回零等待时间和按键保护时间。通过这些常量可以直接看出当前示例的控制参数，例如细分模式固定为 `0x02`，单次移动固定为 `1000` 步。

```
constexpr uint8_t kSteppersSubdivision = 0x02;
constexpr int32_t kStepperMoveSteps = 1000;
constexpr uint32_t kStepperResetWaitMs = 2000;
constexpr uint32_t kButtonStartupGuardMs = 1200;

uint32_t g_button_enable_ms = 0;
```

2. 初始化阶段先调用 `arm.begin()` 完成板级对象初始化，然后通过 `set_subdivision()` 设置滑轨细分模式，并立即执行一次 `reset()` 回零。这里加入了固定延时，目的是给滑轨留出足够的回零时间，避免后续动作过早开始。

```
arm.begin();

arm.board.stepper.set_subdivision(kSteppersSubdivision);
arm.board.stepper.reset();
delay(kStepperResetWaitMs);
```

3. 完成滑轨初始化后，程序会清除按键残留状态并设置按键保护时间，然后把当前模式信息和按键说明显示到 OLED 上。这样上电后界面就已经准备好，用户可以直接看到“BOOT 运行、USER 回零”的控制方式。

```

arm.board.button.update();
arm.board.button.is_clicked(0);
arm.board.button.is_clicked(1);
g_button_enable_ms = millis() + kButtonStartupGuardMs;

arm.board.oled.set_custom_text(0, "Stepper Control");
arm.board.oled.set_custom_text(1, "Subdivision: 0x02");
arm.board.oled.set_custom_text(2, "Move: 1000 steps");
arm.board.oled.set_custom_text(3, "BOOT run USER home");
arm.board.oled.show_custom();

```

4. 主循环中的控制逻辑很直接：按下 `BOOT` 键时调用 `stepper.move(kStepperMoveSteps)` 运行固定步数；按下 `USER` 键时调用 `stepper.reset()` 让滑轨回到初始位置。每次动作发生后，还会同步更新 OLED 的提示信息。

```

if (millis() >= g_button_enable_ms && arm.board.button.is_clicked(0)) {
    arm.board.stepper.move(kStepperMoveSteps);
    arm.board.oled.set_custom_text(3, "BOOT moved 1000");
}

if (millis() >= g_button_enable_ms && arm.board.button.is_clicked(1)) {
    arm.board.stepper.reset();
    arm.board.oled.set_custom_text(3, "USER reset home");
}

```

5. 这份程序把“细分模式设置”和“运动控制”分成了两个阶段。细分模式只在初始化时设置一次，后续无论执行移动还是回零，都不再重复修改该参数。这样可以避免在运行过程中频繁改动驱动配置，保证滑轨动作始终基于同一组步进参数执行。

```

arm.board.stepper.set_subdivision(kSteppersSubdivision);
arm.board.stepper.reset();
delay(kStepperResetWaitMs);

```

6. 主循环末尾持续调用 `arm.board.oled.show_custom()`，因此滑轨动作期间界面会一直保持在当前自定义页面上。配合按键触发后改写第 4 行文本的方式，用户能直接看到最近一次执行的是“移动 1000 步”还是“回到初始位置”。

```

void system_loop_handler(void)
{
    arm.board.button.update();
    arm.board.buzzer.update();
    arm.board.bat.update();

    if (millis() >= g_button_enable_ms && arm.board.button.is_clicked(0)) {
        arm.board.stepper.move(kStepperMoveSteps);
        arm.board.oled.set_custom_text(3, "BOOT moved 1000");
    }

    if (millis() >= g_button_enable_ms && arm.board.button.is_clicked(1)) {
        arm.board.stepper.reset();
        arm.board.oled.set_custom_text(3, "USER reset home");
    }
}

```

```
}

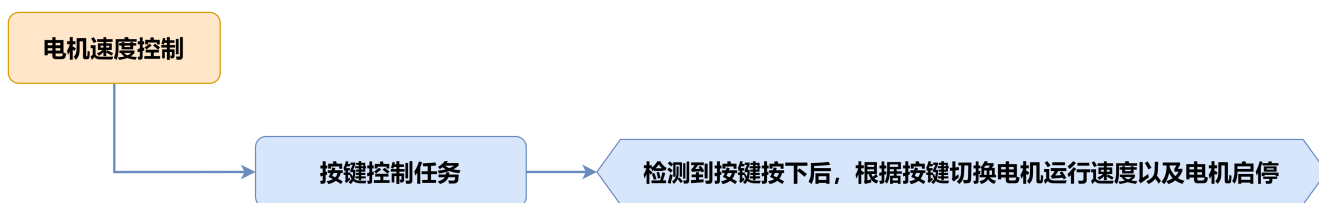
arm.board.oled.show_custom();
}
```

7.6 电机速度控制

7.6.1 项目介绍

本节课程演示如何通过控制板上的电机接口切换电机速度。程序将电机速度分为 `Slow`、`Medium` 和 `Fast` 三档，通过 `USER` 键循环切换速度档位，通过 `BOOT` 键控制电机启动与停止。

7.6.2 实现流程



7.6.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“电机速度控制”程序进行下载，具体下载步骤可参考[7.2 程序下载说明（必看）](#)。

7.6.4 实现效果

程序上电后电机默认停止，OLED 显示当前速度档位为 `Medium`。按下 `USER` 键后，速度档位会在 `Slow`、`Medium` 和 `Fast` 之间循环切换；按下 `BOOT` 键后，电机按照当前档位运行，再次按下则停止。

7.6.5 程序分析

1. 程序先通过 `MotorSpeedProfile` 定义速度档位结构，并用数组保存三组速度参数。与此同时，还定义了当前速度索引、按键使能时间和电机运行状态，用来描述“当前是哪一档”和“当前是否正在输出速度”。

```
struct MotorSpeedProfile {
    const char* name;
    int8_t speed;
};

constexpr MotorSpeedProfile kSpeedProfiles[] = {
    {"Slow", 60},
    {"Medium", 100},
    {"Fast", 127},
};

uint8_t g_speed_profile_index = 1;
uint32_t g_button_enable_ms = 0;
bool g_motor_running = false;
```

2. `apply_motor_output()` 会根据当前是否运行，决定是输出当前档位速度还是输出 0。

`refresh_oled_status()` 则把当前速度档位、运行状态和最近一次动作提示显示到 OLED 上，因此界面始终和实际控制状态保持一致。

```
void apply_motor_output()
{
    int8_t speed = g_motor_running ? kSpeedProfiles[g_speed_profile_index].speed : 0;
    arm.board.motor.set_speed(speed, speed, speed, speed);
}

void refresh_oled_status(const char* action_text)
{
    arm.board.oled.set_custom_text(0, "Motor Speed Ctrl");
    arm.board.oled.set_custom_text(1, String("speed: ") +
    kSpeedProfiles[g_speed_profile_index].name);
    arm.board.oled.set_custom_text(2, g_motor_running ? "Motor: Running" : "Motor:
    Stopped");
    arm.board.oled.set_custom_text(3, action_text);
    arm.board.oled.show_custom();
}
```

3. 初始化阶段先调用 `arm.begin()`，然后立即执行一次 `apply_motor_output()`，确保上电后电机处于停止状态。随后清除按键残留状态并设置保护时间，最后在 OLED 上显示按键功能提示。

```
arm.begin();

apply_motor_output();

arm.board.button.update();
arm.board.button.is_clicked(0);
arm.board.button.is_clicked(1);
g_button_enable_ms = millis() + kButtonStartupGuardMs;

refresh_oled_status("BOOT run / USER speed");
```

4. `toggle_motor_state()` 负责切换电机启动与停止，`switch_speed_profile()` 负责切换速度档位。主循环中根据按键分别调用这两个函数，就构成了完整的“BOOT 控制运行、USER 控制速度”的交互方式。

```

void toggle_motor_state()
{
    g_motor_running = !g_motor_running;
    apply_motor_output();
    refresh_oled_status(g_motor_running ? "BOOT start motor" : "BOOT stop motor");
}

void switch_speed_profile()
{
    g_speed_profile_index = (g_speed_profile_index + 1) % (sizeof(kSpeedProfiles) /
sizeof(kSpeedProfiles[0]));
    apply_motor_output();
    refresh_oled_status("USER switched speed");
}

```

5. 电机输出采用 `set_speed(speed, speed, speed, speed)` 的形式，把四路电机统一设置为同一速度。这意味着示例的重点并不是差速控制或方向控制，而是演示“如何切换速度档位并把档位值输出到电机接口”。

```

void apply_motor_output()
{
    int8_t speed = g_motor_running ? kSpeedProfiles[g_speed_profile_index].speed : 0;
    arm.board.motor.set_speed(speed, speed, speed, speed);
}

```

6. 主循环除了响应两个按键外，还保留了蜂鸣器、电池和 OLED 的常规刷新逻辑，因此这份程序并不是按一次键才更新一次界面，而是始终在后台维持一套基础外设轮询结构，这样速度档位和运行状态的显示才会持续保持最新。

```

void system_loop_handler(void)
{
    arm.board.button.update();
    arm.board.buzzer.update();
    arm.board.bat.update();

    uint32_t now = millis();

    if (now >= g_button_enable_ms && arm.board.button.is_clicked(0)) {
        toggle_motor_state();
    }

    if (now >= g_button_enable_ms && arm.board.button.is_clicked(1)) {
        switch_speed_profile();
    }

    arm.board.oled.show_custom();
}

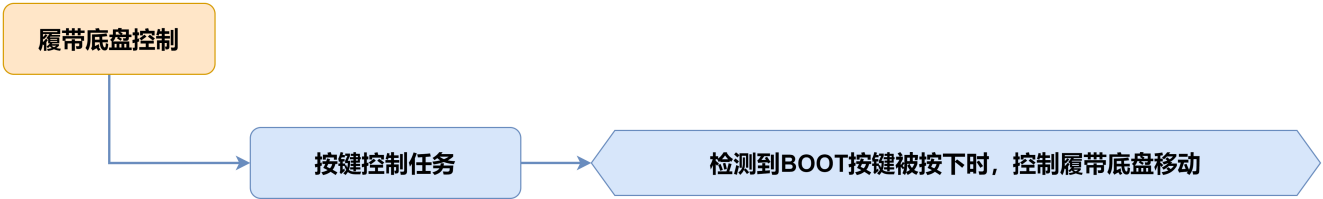
```

7.7 履带底盘控制

7.7.1 项目介绍

本节课程演示如何控制履带底盘完成前进、后退和原地自旋等动作。按下 `BOOT` 键后，履带底盘会依次执行前进 2 秒、后退 2 秒、原地左自旋 3 秒和原地右自旋 3 秒。

7.7.2 实现流程



7.7.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“履带底盘控制”程序进行下载，具体下载步骤可参考[7.2 程序下载说明（必看）](#)。

7.7.4 实现效果

程序上电后，履带底盘保持停止状态。按下 `BOOT` 键后，底盘按固定顺序执行前进、后退、左自旋和右自旋动作，动作完成后自动停止。OLED 会同步显示当前正在执行的动作。

7.7.5 程序分析

1. 程序先定义了按键保护时间、履带前后移动速度和原地旋转速度，这些常量直接决定了各个动作阶段的输出强度。底盘动作示例不使用复杂数据结构，而是通过固定常量让动作流程更直观。

```
constexpr uint32_t kButtonStartupGuardMs = 1200;
constexpr int16_t kTankMoveSpeed = 70;
constexpr int16_t kTankTurnSpeed = 70;

uint32_t g_button_enable_ms = 0;
```

2. `refresh_oled_status()` 负责更新底盘动作提示界面，`run_tank_sequence()` 则按顺序串起完整动作流程。函数内部先后调用 `arm.board.tank.run()` 执行不同动作，并通过 `delay()` 保持指定动作时间。

```
void refresh_oled_status(const char* line3)
{
    arm.board.oled.set_custom_text(0, "Tank Chassis Ctrl");
    arm.board.oled.set_custom_text(1, "BOOT runs action");
    arm.board.oled.set_custom_text(2, "FWD/BWD/SPIN demo");
    arm.board.oled.set_custom_text(3, line3);
    arm.board.oled.show_custom();
}

void run_tank_sequence()
{

```

```

refresh_oled_status("Forward 2s");
arm.board.tank.run(kTankMoveSpeed, 0);
delay(2000);
}

```

3. 初始化阶段先调用 `arm.begin()`，然后通过 `arm.board.tank.stop()` 保证底盘静止，再清空一次按键点击状态并设置上电后的按键保护时间。OLED 随后显示“按下 BOOT 执行动作”的待机提示。

```

arm.begin();
arm.board.tank.stop();

arm.board.button.update();
arm.board.button.is_clicked(0);
arm.board.button.is_clicked(1);
g_button_enable_ms = millis() + kButtonStartupGuardMs;

refresh_oled_status("Press BOOT to run");

```

4. 主循环中的逻辑较为简单：持续更新按键、蜂鸣器和电池状态；一旦检测到 `BOOT` 按键点击，就调用 `run_tank_sequence()` 完成整套动作。动作结束后程序会主动调用 `arm.board.tank.stop()` 停止底盘，并刷新 OLED 提示。

```

if (millis() >= g_button_enable_ms && arm.board.button.is_clicked(0)) {
    run_tank_sequence();
}

arm.board.tank.stop();
refresh_oled_status("Sequence finished");

```

5. 履带底盘动作使用的是 `run(linear, turn)` 这种接口形式，因此前进和后退阶段通过修改第一个参数实现，原地自旋阶段则把线速度设为 `0`，只保留转向速度。这样可以清楚地区分“直线运动”和“原地旋转”在控制参数上的差异。

```

arm.board.tank.run(kTankMoveSpeed, 0);
arm.board.tank.run(-kTankMoveSpeed, 0);
arm.board.tank.run(0, kTankTurnSpeed);
arm.board.tank.run(0, -kTankTurnSpeed);

```

6. `run_tank_sequence()` 把动作执行和界面提示绑在了一起，每进入一个新阶段都会先刷新 OLED 文本，再启动对应动作。这样当底盘处于连续动作过程中时，屏幕显示的内容就能准确对应当前执行到的那一步。

```
void run_tank_sequence()
{
    refresh_oled_status("Forward 2s");
    arm.board.tank.run(kTankMoveSpeed, 0);
    delay(2000);

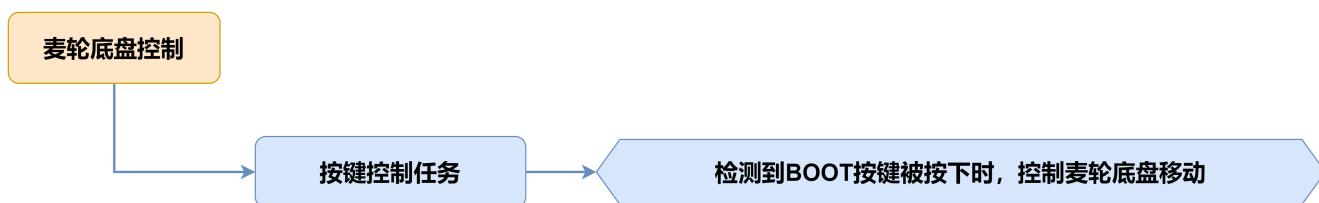
    refresh_oled_status("Backward 2s");
    arm.board.tank.run(-kTankMoveSpeed, 0);
    delay(2000);
}
```

7.8 麦轮底盘控制

7.8.1 项目介绍

本节课程演示如何控制麦轮底盘完成前进、后退、横向平移和原地自旋等动作。按下 **BOOT** 键后，底盘会依次执行前进 2 秒、后退 2 秒、左平移 2 秒、右平移 2 秒、左自旋 3 秒和右自旋 3 秒。

7.8.2 实现流程



7.8.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“麦轮底盘控制”程序进行下载，具体下载步骤可参考[7.2 程序下载说明（必看）](#)。

7.8.4 实现效果

程序上电后，麦轮底盘保持静止。按下 **BOOT** 键后，底盘依次完成前进、后退、左平移、右平移、左自旋和右自旋动作，结束后自动停止。OLED 会同步显示当前动作阶段。

7.8.5 程序分析

1. 程序先定义了按键保护时间、麦轮平移速度和原地旋转速度。由于麦轮底盘的控制依赖 `vx`、`vy`、`wz` 三个量，因此这里分别定义移动速度和旋转速度，后续动作就可以直接复用这些参数。

```
constexpr uint32_t kButtonStartupGuardMs = 1200;
constexpr int16_t kMecanumMoveSpeed = 70;
constexpr int16_t kMecanumSpinsSpeed = 70;

uint32_t g_button_enable_ms = 0;
```

2. `refresh_oled_status()` 用于更新 OLED 动作界面，`stop_mecanum()` 用于统一停止底盘，`run_mecanum_sequence()` 则把所有动作阶段按顺序排好。每进入一个新阶段，程序都会先更新 OLED，再调用 `set_velocity(vx, vy, wz)` 输出对应底盘速度。

```

void stop_mecanum()
{
    arm.board.mecanum.set_velocity(0, 0, 0);
}

void run_mecanum_sequence()
{
    refresh_oled_status("Forward 2s");
    arm.board.mecanum.set_velocity(kMecanumMoveSpeed, 0, 0);
    delay(2000);
}

```

3. 初始化阶段先调用 `arm.begin()`，然后立刻执行一次 `stop_mecanum()`，保证上电时底盘静止。之后清理按键状态并设置按键保护时间，最后在 OLED 上显示待机提示。

```

arm.begin();
stop_mecanum();

arm.board.button.update();
arm.board.button.is_clicked(0);
arm.board.button.is_clicked(1);
g_button_enable_ms = millis() + kButtonStartupGuardMs;

refresh_oled_status("Press BOOT to run");

```

4. 主循环中持续更新按键、蜂鸣器和电池状态，当检测到 `BOOT` 按键点击后，程序调用 `run_mecanum_sequence()` 执行完整动作链。动作执行完成后，再通过 `stop_mecanum()` 让底盘回到停止状态，并刷新 OLED 提示。

```

if (millis() >= g_button_enable_ms && arm.board.button.is_clicked(0)) {
    run_mecanum_sequence();
}

stop_mecanum();
refresh_oled_status("Sequence finished");

```

5. 麦轮底盘的动作接口是 `set_velocity(vx, vy, wz)`，所以前进和后退主要改变 `vx`，左右平移主要改变 `vy`，原地自旋主要改变 `wz`。这和履带底盘只使用线速度、转向速度的形式不同，也正好体现了麦轮底盘在运动自由度上的优势。

```

arm.board.mecanum.set_velocity(kMecanumMoveSpeed, 0, 0);
arm.board.mecanum.set_velocity(-kMecanumMoveSpeed, 0, 0);
arm.board.mecanum.set_velocity(0, kMecanumMoveSpeed, 0);
arm.board.mecanum.set_velocity(0, -kMecanumMoveSpeed, 0);
arm.board.mecanum.set_velocity(0, 0, kMecanumSpinSpeed);

```

6. 程序额外封装了 `stop_mecanum()`，把停止底盘的操作统一收口到一个函数里。这样无论是在初始化阶段还是动作结束阶段，都调用同一个停止接口，后续如果需要调整停止方式，也只需要改动这一处。

```

void stop_mecanum()
{
    arm.board.mecanum.set_velocity(0, 0, 0);
}

void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    setCpuFrequencyMhz(240);
    Serial.begin(115200);

    arm.begin();
    stop_mecanum();
}

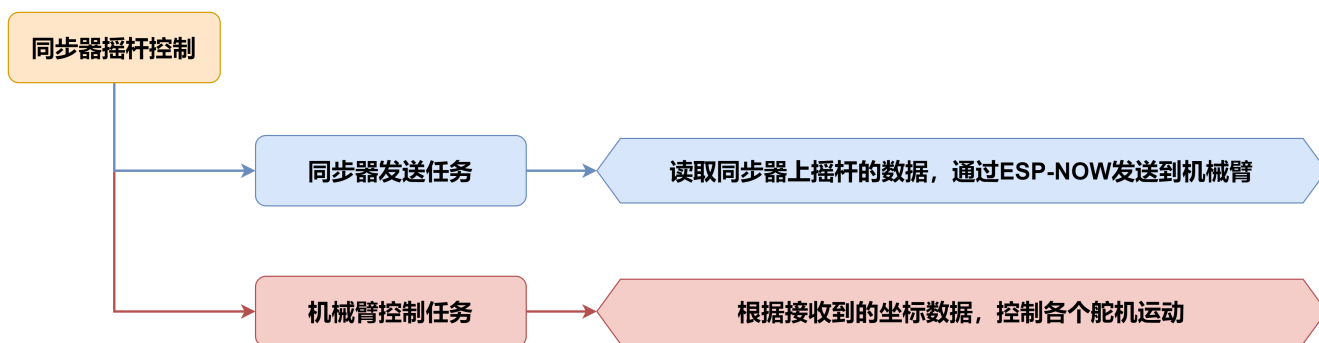
```

7.9 同步器摇杆控制

7.9.1 项目介绍

本节课程演示如何通过同步器上的双轴摇杆，借助 ESP-NOW 无线链路对机械臂进行实时坐标控制。同步器程序负责读取摇杆 X、Y 两路模拟量，将其映射为机械臂末端的 X、Z 目标坐标，并打包为 `CMD_COORDINATE_SET` 指令周期发送给机械臂；机械臂程序负责接收该坐标控制指令，透传给 AT32 下位机执行逆运动学与插补运动，同时在 OLED 上显示当前末端位姿。

7.9.2 实现流程



7.9.3 程序下载

- 机械臂程序下载

找到本文档同目录下的“02 程序源码”文件夹，打开“同步器摇杆控制\机械臂\Nex_Arm”中的程序文件，将其下载到机械臂控制板中。具体下载步骤可参考 [7.2 程序下载说明（必看）](#)。

- 同步器程序下载

找到本文档同目录下的“02 程序源码”文件夹，打开“同步器摇杆控制\同步器\Nex_Arm”中的程序文件，将其下载到同步器控制板中。具体下载步骤可参考 [7.2 程序下载说明（必看）](#)。

7.9.4 实现效果

两端程序分别下载完成后，上电进入相同的 ESP-NOW 信道。同步器端会持续读取摇杆位置，并把对应的机械臂目标坐标通过 ESP-NOW 发给机械臂端；机械臂端收到坐标控制帧后，会带动末端在 X、Z 平面内随摇杆方向变化而移动。同步器串口会输出当前目标坐标，机械臂 OLED 会实时显示当前末端位姿。

7.9.5 程序分析

• 机械臂分析

1. 机械臂端先定义了 ESP-NOW 接收控制所需的运行时状态，包括当前信道、全局加速度、同步模式开关、OLED 刷新时间、电池刷新时间以及当前位姿缓存等。这样程序在接收到坐标控制指令后，既可以转发到底层执行，也可以同步更新本地显示状态。

```
CommProtocol_t at32_protocol;
Preferences prefs;

uint8_t global_channel = DEFAULT_ESPNOW_CHANNEL;
uint8_t global_acc = 245;
bool espnow_sync_enabled = false;

uint32_t last_oled_refresh_ms = 0;
uint32_t last_bat_refresh_ms = 0;
uint32_t button_enable_ms = 0;
```

2. `func_ctrl_callback()` 是机械臂端处理 ESP-NOW 协议数据的核心入口。当收到 `CMD_COORDINATE_SET` 时，程序会先把该指令透传给 AT32，再把参数中的 X、Y、Z、Pitch、Roll、Claw 更新到 `arm.current_pose` 中，便于 OLED 页面同步显示。

```
case CMD_COORDINATE_SET:
    servo.tx_frame_write(0xFF, self->elements.cmd, self->elements.args, payload_len);
    update_pose_from_coordinate_set(self->elements.args, payload_len);
    break;
```

3. `update_pose_from_coordinate_set()` 负责把收到的 14 字节坐标参数拆成具体位姿值。这里按照协议顺序依次读取 Pitch、X、Y、Z、Roll、Claw，并把解析结果写入 `arm.current_pose`，因此 OLED 显示的内容与最近一次收到的坐标控制指令是一致的。

```
void update_pose_from_coordinate_set(const uint8_t* args, uint8_t payload_len)
{
    int16_t p_raw = (int16_t)((args[1] << 8) | args[0]);
    int16_t x_raw = (int16_t)((args[3] << 8) | args[2]);
    int16_t y_raw = (int16_t)((args[5] << 8) | args[4]);
    int16_t z_raw = (int16_t)((args[7] << 8) | args[6]);

    arm.current_pose.x = (float)x_raw;
    arm.current_pose.y = (float)y_raw;
    arm.current_pose.z = (float)z_raw;
    arm.current_pose.pitch = (float)p_raw / 10.0f;
}
```

4. 主循环里先处理挂起的同步帧和 AT32 回包，再进行按键、电池和 OLED 的周期刷新。虽然这一节主要展示摇杆坐标控制，但机械臂端仍保留了完整的基础刷新链路，因此在接收控制过程中仍能稳定维护界面和状态显示。

```
void system_loop_handler(void)
{
    if (espnow_sync_enabled) {
        espnow_ctrl.processPendingSyncFrame(0);
    }

    pump_at32_feedback();
    arm.board.button.update();
    arm.board.buzzer.update();
    arm.board.bat.update();
}
```

5. OLED 刷新部分按固定周期显示 X、Y、Z、Pitch、Roll、Claw 六项末端位姿数据。这样在摇杆推动过程中，可以直观看到末端坐标变化与控制指令之间的对应关系，便于验证无线控制效果是否正确。

```
if (now - last_oled_refresh_ms >= OLED_REFRESH_INTERVAL_MS) {
    last_oled_refresh_ms = now;
    arm.board.oled.show_status(
        arm.current_pose.x,
        arm.current_pose.y,
        arm.current_pose.z,
        arm.current_pose.pitch,
        arm.current_pose.roll,
        arm.current_pose.claw
    );
}
```

• 同步器分析

1. 同步器端先定义了摇杆控制所需的核心变量，包括广播目标 MAC、发送周期、当前目标坐标、摇杆中心值以及调试输出时间等。这些变量决定了发送频率、坐标范围和摇杆校准结果。

```
uint8_t broadcastMac[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
unsigned long lastSendTime = 0;
unsigned long lastDebugTime = 0;

float current_x = 200.0f;
float current_y = 0.0f;
float current_z = 200.0f;
int joy_center_x = 2048;
int joy_center_y = 2048;
```

2. `calibrateJoystickCenter()` 会在启动时读取 20 次摇杆模拟量求平均值，用来得到当前摇杆的真实中心点。这样可以减少摇杆零位偏差带来的漂移问题，使摇杆松手后更容易保持在中间坐标附近。

```

void calibrateJoystickCenter() {
    long sumX = 0;
    long sumY = 0;
    for (int i = 0; i < 20; i++) {
        sumX += analogRead(JOYSTICK_X_PIN);
        sumY += analogRead(JOYSTICK_Y_PIN);
        delay(2);
    }
    joy_center_x = sumX / 20;
    joy_center_y = sumY / 20;
}

```

3. 初始化阶段先配置摇杆引脚，再完成 ESP-NOW 初始化、设置信道和添加广播节点。由于使用的是广播地址，摇杆同步器发出的坐标控制包可以被同一信道内的目标机械臂直接接收，无需预先写死对端单独的 MAC 地址。

```

WiFi.mode(WIFI_STA);
WiFi.disconnect();

esp_wifi_set_promiscuous(true);
esp_wifi_set_channel(WIFI_CHANNEL, WIFI_SECOND_CHAN_NONE);
esp_wifi_set_promiscuous(false);

esp_now_peer_info_t peerInfo = {};
memcpy(peerInfo.peer_addr, broadcastMac, 6);
peerInfo.channel = WIFI_CHANNEL;
peerInfo.encrypt = false;
esp_now_add_peer(&peerInfo);

```

4. 主循环中每隔 `SEND_INTERVAL_MS` 读取一次摇杆 X、Y 两路模拟量，并根据中心值和死区范围把它们映射到机械臂目标 X、Z 坐标。这里 Y 坐标保持为 0，因此摇杆控制的是一个二维平面内的移动过程。

```

if (abs(joyY - joy_center_y) < 100) {
    joyY = joy_center_y;
}
if (joyY <= joy_center_y) {
    current_x = map(joyY, 0, joy_center_y, 400, 200);
} else {
    current_x = map(joyY, joy_center_y, 4095, 200, 400);
}

```

5. 摇杆坐标计算完成后，程序会把 `Pitch`、`X`、`Y`、`Z`、`Roll`、`Claw`、`Time` 依次打包成 `CMD_COORDINATE_SET` 坐标控制帧，并计算校验和后通过 `esp_now_send()` 广播发送。这样机械臂端收到的就是一条标准串口协议格式的坐标控制命令。

```

packet[0] = 0xFF;
packet[1] = 0xFF;
packet[2] = 0xFF;
packet[3] = 0x10;
packet[4] = 0x08;

packet[7] = x & 0xFF;
packet[8] = (x >> 8) & 0xFF;
packet[11] = z & 0xFF;
packet[12] = (z >> 8) & 0xFF;

esp_now_send(broadcastMac, packet, sizeof(packet));

```

6. 同步器端保留了串口调试输出，每隔一段时间打印当前目标 `x`、`y`、`z` 坐标。通过这些打印信息，可以直接观察摇杆变化和发送坐标之间的对应关系，便于调试控制方向、死区范围和坐标映射区间是否合理。

```

if (currentMillis - lastDebugTime > 200) {
    lastDebugTime = currentMillis;
    Serial.printf("Joy: x=%.1f y=%.1f z=%.1f\n", current_x, current_y, current_z);
}

```

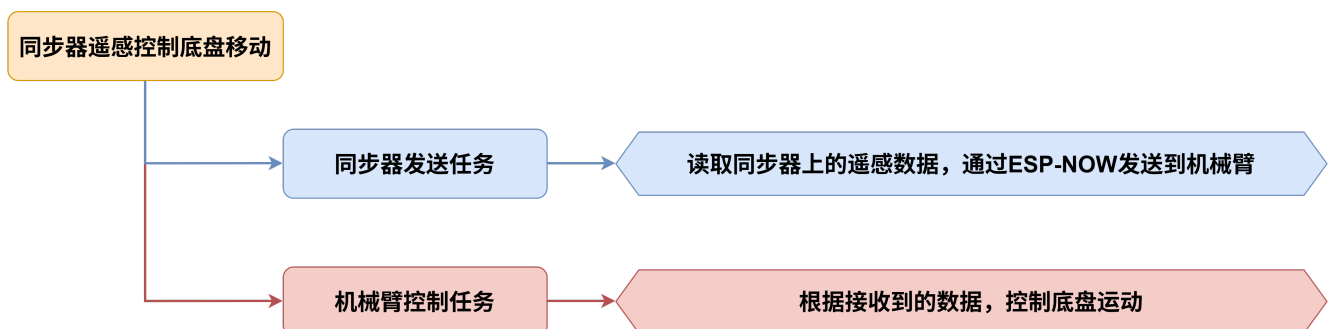
7.10 同步器摇杆控制移动底盘

7.10.1 项目介绍

本节课程演示如何通过同步器上的双轴摇杆，借助 ESP-NOW 无线链路控制移动底盘运动。同步器程序在保留原有机械臂示教同步功能的基础上，增加了摇杆底盘控制模式：按下摇杆按键后进入底盘控制模式，程序会读取摇杆 X、Y 两路模拟量，将其转换为底盘速度指令，并通过 ESP-NOW 周期发送给接收端；再次按下摇杆按键后退出底盘控制模式，同时发送停止指令，使移动底盘停止运动。

该程序默认使用麦轮底盘控制方式，摇杆前后方向对应底盘前进、后退，摇杆左右方向对应底盘旋转控制。若需要使用履带底盘，也可以通过修改程序中的 `CHASSIS_CONTROL_MODE` 宏切换为坦克底盘控制方式。

7.10.2 实现流程



7.10.3 程序下载

- 机械臂与移动底盘接收端程序下载

找到本文档同目录下的“02 程序源码”文件夹，打开“同步器遥感控制底盘程序\机械臂\Nex_Arm”接收端程序，将其下载到机械臂或移动底盘控制板中。该接收端需要开启 ESP-NOW 接收功能，并与同步器端保持相同的 Wi-Fi 信道。具体下载步骤可参考 [7.2 程序下载说明（必看）](#)。

- 同步器程序下载

找到本文档同目录下的“02 程序源码”文件夹，打开“同步器遥感控制底盘程序\同步器\Nex_Arm_sys_Chassis_Joystick”中的 `Nex_Arm_sys_Chassis_Joystick.ino` 程序文件，将其下载到同步器控制板中。具体下载步骤可参考 [7.2 程序下载说明（必看）](#)。

Note

本工程目录中的 `Nex_Arm_sys_Chassis_Joystick` 主要是同步器发送端程序，负责读取摇杆和舵机位置并通过 ESP-NOW 广播发送。移动底盘接收端需要在普通 `Nex_Arm` 程序中完成 ESP-NOW 接收、协议解析和底盘电机执行，且必须与同步器端保持相同信道和相同命令格式。

7.10.4 实现效果

两端程序分别下载完成后，上电进入相同的 ESP-NOW 信道。同步器端会继续读取 1~6 号舵机位置，并周期广播机械臂示教同步数据。按下摇杆按键后，同步器进入底盘摇杆控制模式，此时推动摇杆即可控制移动底盘运动：摇杆前后控制底盘前进和后退，摇杆左右控制底盘旋转。再次按下摇杆按键后，同步器退出底盘控制模式，并向底盘发送停止指令。

同步器串口会周期打印当前读取到的 6 路舵机位置，同时显示底盘控制模式是否开启。通过串口输出中的 `chassis=ON` 或 `chassis=OFF`，可以判断当前是否处于摇杆控制底盘模式。遥感对应底盘运动方向如下图：



7.10.5 程序分析

- 同步器分析

1. 程序开头先定义 ESP-NOW 信道、发送周期、底盘控制命令、摇杆引脚和底盘控制模式等参数。其中 `SEND_INTERVAL_MS` 用于控制机械臂示教数据发送周期，`CHASSIS_SEND_INTERVAL_MS` 用于控制底盘摇杆指令发送周期。默认情况下，底盘控制模式为麦轮底盘模式。

```
#define WIFI_CHANNEL      4
#define SEND_INTERVAL_MS  4
#define CHASSIS_SEND_INTERVAL_MS 20
#define CMD_MECANUM_CONTROL 34
#define CMD_TANK_CONTROL  35

#define JOYSTICK_SW_PIN   4
#define JOYSTICK_X_PIN    39
#define JOYSTICK_Y_PIN    36

#define CHASSIS_MODE_MECANUM 1
#define CHASSIS_MODE_TANK    2
#define CHASSIS_CONTROL_MODE CHASSIS_MODE_MECANUM
```

2. 程序还定义了摇杆死区、最大速度和方向反转参数。`JOYSTICK_DEADZONE` 用于过滤摇杆中心附近的小幅抖动，避免摇杆松手后底盘仍缓慢移动；`JOYSTICK_MAX_SPEED` 用于限制输出速度范围；`JOYSTICK_INVERT_X` 和 `JOYSTICK_INVERT_Y` 用于调整摇杆方向和底盘运动方向之间的对应关系。

```
#define JOYSTICK_DEADZONE    120
#define JOYSTICK_MAX_SPEED   60
#define JOYSTICK_INVERT_X    1
#define JOYSTICK_INVERT_Y    1

bool isJoystickMode = false;
int lastBtnState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 200;

int joy_center_x = 2048;
int joy_center_y = 2048;
```

3. `ArmPacket_t` 是原有同步器示教功能使用的数据结构，其中 `seq` 保存当前时间序号，`pos[6]` 保存 1~6 号舵机当前位置。也就是说，本程序并不是只发送底盘控制指令，而是在底盘控制模式开启后，同时保留机械臂示教同步数据发送。

```
typedef struct __attribute__((packed)) {
    uint32_t seq;
    int16_t pos[6];
} ArmPacket_t;

uint8_t broadcastMac[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
int16_t last_valid_pos[6] = {2048, 2048, 2048, 2048, 2048, 2048};
```

4. 初始化阶段先启动串口、配置摇杆引脚、初始化舵机总线，然后将 Wi-Fi 设置为 STA 模式，并设置 ESP-NOW 工作信道。随后程序初始化 ESP-NOW，并添加广播地址作为发送目标。由于使用广播地址，同一信道内的接收端都可以收到同步器发送的数据。

```
Serial.begin(1000000);
setCpuFrequencyMhz(240);

pinMode(JOYSTICK_SW_PIN, INPUT_PULLUP);
pinMode(JOYSTICK_X_PIN, INPUT);
pinMode(JOYSTICK_Y_PIN, INPUT);

servo.begin(Serial1, 1000000, 17, 16);
WiFi.mode(WIFI_STA);
WiFi.disconnect();
esp_wifi_set_promiscuous(true);
esp_wifi_set_channel(WIFI_CHANNEL, WIFI_SECOND_CHAN_NONE);
esp_wifi_set_promiscuous(false);
```

5. ESP-NOW 初始化完成后，程序把广播 MAC 地址添加为通信节点，并关闭 1~6 号舵机力矩。关闭力矩后，同步器机械臂可以被手动拖动，程序再读取各舵机当前位置并广播给接收端，从而保留原有“手拖示教、无线同步”的功能。

```

if (esp_now_init() != ESP_OK) {
    return;
}

esp_now_peer_info_t peerInfo = {};
memcpy(peerInfo.peer_addr, broadcastMac, 6);
peerInfo.channel = WIFI_CHANNEL;
peerInfo.encrypt = false;
esp_now_add_peer(&peerInfo);

for(int i=1; i<=6; i++) {
    servo.disable_torque(i);
    delay(5);
}

```

6. 主循环中首先处理 USB 串口收到的协议数据。该部分主要用于 LeRobot 或舵机协议命令处理，与摇杆控制底盘不是同一条主链路，但它可以同步器在运行 ESP-NOW 示教同步的同时，继续响应上位机通过 USB 发来的部分控制或查询指令。

```

while (Serial.available()) {
    uint8_t buf[64];
    int n = Serial.readBytes(buf, min((int)Serial.available(), (int)64));
    usb_protocol.parsing(buf, n);
}

```

7. 摇杆按键接在 GPIO4，程序通过 `digitalRead(JOYSTICK_SW_PIN)` 检测按键状态。当检测到按键从松开变为按下，并且超过消抖时间后，就切换一次 `isJoystickMode` 状态。进入摇杆控制模式时，程序会连续读取 20 次摇杆 X、Y 模拟量并求平均值，作为当前摇杆中心点，减少摇杆零位偏差带来的漂移。

```

int reading = digitalRead(JOYSTICK_SW_PIN);
if (reading == LOW && lastBtnState == HIGH && (currentMillis - lastDebounceTime >
debounceDelay)) {
    isJoystickMode = !isJoystickMode;
    lastDebounceTime = currentMillis;
    if (isJoystickMode) {
        long sumX = 0;
        long sumY = 0;
        for (int i = 0; i < 20; i++) {
            sumX += analogRead(JOYSTICK_X_PIN);
            sumY += analogRead(JOYSTICK_Y_PIN);
            delay(2);
        }
        joy_center_x = sumX / 20;
        joy_center_y = sumY / 20;
        beep(1000, 100);
    }
}

```

8. 当再次按下摇杆按键退出底盘控制模式时，程序会调用 `send_chassis_stop()` 主动发送停止指令。默认麦轮模式下发送 3 个速度值 0, 0, 0；如果切换为坦克底盘模式，则发送 2 个速度值 0, 0。这样可以避免退出模式后底盘保持最后一次速度继续运动。

```
static void send_chassis_stop() {
    #if CHASSIS_CONTROL_MODE == CHASSIS_MODE_TANK
        uint8_t data[2] = {0, 0};
        send_protocol_packet(CMD_TANK_CONTROL, data, sizeof(data));
    #else
        uint8_t data[3] = {0, 0, 0};
        send_protocol_packet(CMD_MECANUM_CONTROL, data, sizeof(data));
    #endif
}
```

9. `joystick_axis_to_speed()` 用于把摇杆模拟量转换为底盘速度。程序先计算当前摇杆值与中心值之间的偏差，如果偏差处于死区范围内，则速度直接置为 0；如果超过死区，则根据偏移方向和偏移量，将 ADC 数值线性映射到 `-JOYSTICK_MAX_SPEED` 到 `JOYSTICK_MAX_SPEED` 的速度范围内。

```
static int8_t joystick_axis_to_speed(int value, int center, bool invert) {
    int delta = value - center;
    if (abs(delta) <= JOYSTICK_DEADZONE) {
        return 0;
    }

    long speed;
    if (delta > 0) {
        int span = max(1, 4095 - center - JOYSTICK_DEADZONE);
        speed = map(delta - JOYSTICK_DEADZONE, 0, span, 0, JOYSTICK_MAX_SPEED);
    } else {
        int span = max(1, center - JOYSTICK_DEADZONE);
        speed = -map(-delta - JOYSTICK_DEADZONE, 0, span, 0, JOYSTICK_MAX_SPEED);
    }

    speed = constrain(speed, -JOYSTICK_MAX_SPEED, JOYSTICK_MAX_SPEED);
    if (invert) {
        speed = -speed;
    }
    return (int8_t)speed;
}
```

10. 当底盘摇杆控制模式开启后，主循环每隔 `CHASSIS_SEND_INTERVAL_MS` 调用一次 `send_chassis_from_joystick()`。该函数读取摇杆 X、Y 两路模拟量，分别转换为 `x_speed` 和 `y_speed`，再根据当前底盘模式打包成不同的数据格式发送。

```
if (isJoystickMode && currentMillis - lastChassisSendTime >= CHASSIS_SEND_INTERVAL_MS) {
    lastChassisSendTime = currentMillis;
    send_chassis_from_joystick();
}
```

11. 默认麦轮模式下，程序发送 `CMD_MECANUM_CONTROL` 指令，数据内容为 `[vx, vy, vz]`。其中 `y_speed` 作为前后方向速度，第二个参数固定为 `0`，`x_speed` 作为旋转方向速度。因此本例中摇杆主要控制麦轮底盘前进、后退和原地转向，并没有启用横向平移速度。

```
static void send_chassis_from_joystick() {
    int joyX = analogRead(JOYSTICK_X_PIN);
    int joyY = analogRead(JOYSTICK_Y_PIN);

    int8_t x_speed = joystick_axis_to_speed(joyX, joy_center_x, JOYSTICK_INVERT_X);
    int8_t y_speed = joystick_axis_to_speed(joyY, joy_center_y, JOYSTICK_INVERT_Y);

    #if CHASSIS_CONTROL_MODE == CHASSIS_MODE_TANK
        uint8_t data[2];
        data[0] = (uint8_t)y_speed;
        data[1] = (uint8_t)x_speed;
        send_protocol_packet(CMD_TANK_CONTROL, data, sizeof(data));
    #else
        uint8_t data[3];
        data[0] = (uint8_t)y_speed;
        data[1] = 0;
        data[2] = (uint8_t)x_speed;
        send_protocol_packet(CMD_MECANUM_CONTROL, data, sizeof(data));
    #endif
}
```

12. 底盘控制数据并不是直接发送裸速度值，而是先由 `send_protocol_packet()` 封装成标准协议帧。协议帧包含帧头、ID、长度、命令字、数据和校验和，最后通过 `esp_now_send()` 广播发送给接收端。麦轮模式下发送的命令字为 `34`，坦克模式下发送的命令字为 `35`。

```
static void send_protocol_packet(uint8_t cmd, const uint8_t *data, uint8_t data_len) {
    uint8_t packet[32];
    uint8_t packet_len = 6 + data_len;

    packet[0] = 0xFF;
    packet[1] = 0xFF;
    packet[2] = 0xFF;
    packet[3] = 2 + data_len;
    packet[4] = cmd;
    for (uint8_t i = 0; i < data_len; i++) {
        packet[5 + i] = data[i];
    }
    packet[5 + data_len] = protocol_checksum(&packet[2], packet[3] + 1);

    esp_now_send(broadcastMac, packet, packet_len);
}
```

13. 以麦轮底盘控制为例，发送的数据帧格式如下。接收端收到该帧后，需要按照相同协议解析命令字和参数，并把 `vx`、`vy`、`vz` 转换为底盘电机速度。

```
FF FF FF 05 22 vx 00 vz checksum
```

其中 0x22 是十进制 34，对应 CMD_MECANUM_CONTROL。由于程序中把 int8_t 速度强制转换为 uint8_t 后发送，负数会以补码形式出现在数据帧中，例如 -10 会发送为 246。因此接收端解析时需要按有符号 8 位整数理解这些速度值。

14. 机械臂示教同步数据仍然由 send_teach_positions() 周期发送。该函数会依次读取 1~6 号舵机当前位置，如果读取成功就更新 last_valid_pos[]，如果读取失败则继续使用上一次有效值。最后程序把 6 路位置写入 ArmPacket_t，并通过 ESP-NOW 广播发送。

```
static void send_teach_positions(unsigned long currentMillis) {
    ArmPacket_t teachPacket;
    teachPacket.seq = currentMillis;

    for(int i = 0; i < 6; i++) {
        int16_t pos = 0;
        ServoStatus_t status = servo.read_pos(i + 1, &pos);
        if (status.error_bits.bit_rx == 0 && status.error_bits.bit_tx == 0 && pos != 0) {
            last_valid_pos[i] = pos;
        }
    }

    for(int i = 0; i < 6; i++) {
        teachPacket.pos[i] = last_valid_pos[i];
    }

    esp_now_send(broadcastMac, (uint8_t*)&teachPacket, sizeof(ArmPacket_t));
}
```

15. 主循环最后用两个不同的周期分别发送底盘控制包和机械臂示教包。底盘控制包只在 isJoystickMode 为 true 时发送，周期为 20ms；机械臂示教包一直发送，周期为 4ms。因此本程序的执行特点是：同步器示教功能持续运行，底盘摇杆控制功能通过按键开关临时叠加。

```
if (isJoystickMode && currentMillis - lastChassisSendTime >= CHASSIS_SEND_INTERVAL_MS) {
    lastChassisSendTime = currentMillis;
    send_chassis_from_joyystick();
}

if (currentMillis - lastSendTime >= SEND_INTERVAL_MS) {
    lastSendTime = currentMillis;
    send_teach_positions(currentMillis);
}
```

16. 程序每隔一段时间会通过串口输出当前 6 路舵机位置和底盘控制模式状态。调试时可以打开串口监视器，如果看到 chassis=ON，说明当前已经进入摇杆控制底盘模式；如果看到 chassis=OFF，说明当前只保留机械臂示教同步发送，底盘摇杆控制没有开启。

```

if (currentMillis - lastDebugTime > 1000) {
    Serial.printf("[POS] %d %d %d %d %d %d | chassis=%s\n",
        last_valid_pos[0], last_valid_pos[1], last_valid_pos[2],
        last_valid_pos[3], last_valid_pos[4], last_valid_pos[5],
        isJoystickMode ? "ON" : "OFF");
    lastDebugTime = currentMillis;
}

```

17. 如果需要调整底盘控制效果，可以优先修改摇杆相关参数。当底盘松手后仍然轻微移动时，可以适当增大 `JOYSTICK_DEADZONE`；当速度过快或不易控制时，可以减小 `JOYSTICK_MAX_SPEED`；当前进、后退或左右方向与预期相反时，可以修改 `JOYSTICK_INVERT_X` 或 `JOYSTICK_INVERT_Y`。

```

#define JOYSTICK_DEADZONE    120
#define JOYSTICK_MAX_SPEED   60
#define JOYSTICK_INVERT_X    1
#define JOYSTICK_INVERT_Y    1

```

18. 如果使用履带底盘，可以把底盘模式改为 `CHASSIS_MODE_TANK`。此时发送的数据变为 `[speed, turn]`，其中 `speed` 由摇杆前后方向控制，`turn` 由摇杆左右方向控制。接收端需要支持 `CMD_TANK_CONTROL` 指令，才能正确执行履带底盘速度控制。

```

#define CHASSIS_CONTROL_MODE CHASSIS_MODE_TANK

```

④ Note

本工程目录中的 `Nex_Arm_sys_Chassis_Joystick` 主要是同步器发送端程序，负责读取摇杆和舵机位置并通过 ESP-NOW 广播发送。移动底盘接收端需要在普通 `Nex_Arm` 程序中完成 ESP-NOW 接收、协议解析和底盘电机执行，且必须与同步器端保持相同信道和相同命令格式。